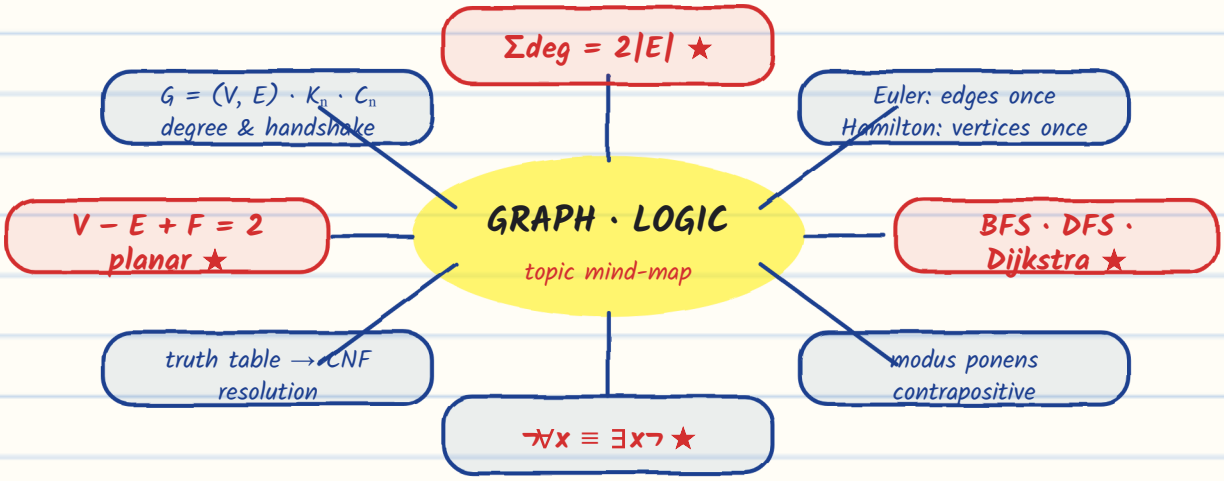
# Graph Theory & Calculus

Σ deg = 2|E| · Euler vs Hamilton · BFS/DFS/Dijkstra · truth tables & quantifiers: exam-ready in 11 sheets




↑ the whole topic on one page: red = high-yield, blue = definitions

## ➜ Syllabus coverage

"Discrete Mathematics, Theory of Computation and Basic Graph Theory: Set theory; Relations; Functions; Graph theory; Propositional calculus and predicate calculus; Mathematical reasoning: induction, contradiction and recursion; ..." Sets, relations and functions are in the previous lesson; induction, counting and algebraic structures have their own lessons in the chapter.

### High-yield list

① Σdeg(v) = **2|E|**; odd vertices in pairs
② $K_n$: deg n−1, edges n(n−1)/2
③ Euler circuit ⇔ every deg **even**
④ Euler path ⇔ exactly **two** odd deg
⑤ Euler = **edges**, Hamilton = **vertices**
⑥ Planar: V − E + F = **2**; E ≤ 3V − 6
⑦ Bipartite planar: E ≤ **2V − 4**
⑧ Tree: n vertices ⇒ **n − 1** edges
⑨ Dijkstra needs weights **≥ 0**
⑩ p → q false only at T, F; p ⊕ q = ¬(p ↔ q)

**Legend** yellow = key term · red circle = must-memorise fact · **purple box** = memory trick · **red box** = exam trap · **green** = worked example

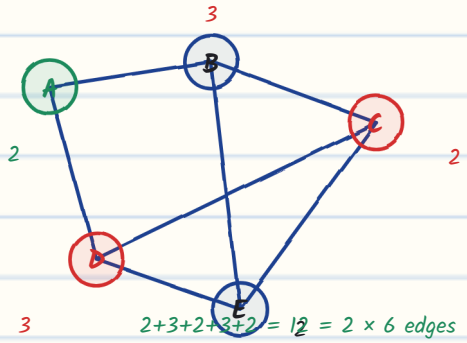
## ➜ Contents

✓ Graphs, vertices, degree ... p. 2
✓ Euler paths & circuits ... p. 3
✓ Hamilton paths & circuits ... p. 3
✓ Bipartite & planar ... p. 4
✓ Trees · BFS & DFS ... p. 5
✓ MST & shortest path ... p. 6
✓ Propositional calculus ... p. 7
✓ CNF & resolution ... p. 8 · Predicate & inference ... p. 9 · Night-before p. 10 · Practice p. 11

# 1 Graph Foundations: Vertices, Edges & Degree

definition ★

**Definition** A **graph** $G = (V, E)$ is an ordered pair of sets: $V$ = the **vertices** (nodes), $E$ = the **edges**, each edge joining exactly two vertices. Write $|V| = n$ (order) and $|E| = m$ (size). A graph is **simple** when it has no loop and no pair of parallel edges; a **multigraph** allows both. An edge is **incident** to a vertex when it has that endpoint.




numbers beside each node are its **degree**; the red ones are **odd-degree**

| Name | What it is |
|---|---|
| **Simple graph** | no loop, no parallel edge; $\deg(v) \le n - 1$ |
| **Complete $K_n$** | every pair joined; deg $n-1$; $\|E\| = n(n-1)/2$ |
| **Cycle $C_n$** | $n$ vertices in a ring; every degree 2 |
| **Path $P_n$** | vertices in a line; exactly **two** of degree 1 |
| **$r$-regular** | every vertex of degree $r$, so $m = nr/2$ |
| **Bipartite** | $V = V_1 \cup V_2$, every edge across the split |

**Two ways to store a graph**

**Adjacency matrix** — an $n \times n$ table, $A[i][j] = 1$ when $(v_i, v_j)$ is an edge: $O(V^2)$ space and build time, and $\deg(v_i)$ is the count of 1s in row $i$.

**Adjacency list** — one neighbour list per vertex, $O(V + E)$: it wins on sparse graphs.

| Term | Exact definition | Closed? |
|---|---|---|
| **Walk** | any alternating sequence of vertices and edges — no restriction at all | may be |
| **Trail** | a walk that uses **no edge twice** (vertices may repeat) | may be |
| **Path** | a walk that uses **no vertex twice** (so no edge twice either) | never |
| **Cycle** | a closed path: same start and end vertex, **length ≥ 3** | must be |

**Memory trick** "**W**alks are **W**ild, **T**rails have **T**ight edges, **P**aths are **P**ure." Each rule removes one freedom — walk ⊂ trail ⊂ path, and a cycle is a path that closed up, so it is the **strongest** of the four.

**Exam trap** (i) A **cycle needs length ≥ 3** in a simple graph — $u, v, u$ is a walk, not a cycle. (ii) "Starts and ends at the same vertex" is only half the definition; no other vertex may repeat. (iii) An isolated vertex has degree 0. (iv) **Isomorphic** needs a bijection preserving edges — same $n$, $m$ and degree sequence is **not** enough.

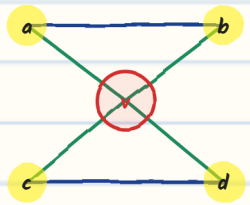
## 2 Euler Paths & Circuits

most tested ★

**Definition** An **Euler trail** uses **every edge exactly once**; an **Euler circuit** is one that also returns to its start. Vertices may repeat as often as needed — **edges** are what count. For a connected undirected graph: **Euler circuit ⇔ every degree even**; **Euler path but no circuit ⇔ exactly two odd degrees**; more than two odd degrees ⇒ no Euler trail at all, and a **disconnected** graph never has one however even its degrees.

| Connected undirected graph | What exists |
|---|---|
| every deg **even** | **Euler circuit** — and therefore also a path |
| exactly **two** odd deg | Euler **path** only; it must start at one odd vertex and end at the other |
| more than two odd deg | **no** Euler trail |

Euler circuit ✓ Hamilton circuit ✗




deg: v = 4, a = b = c = d = 2 → all even

two triangles glued at **v**: all degrees even, so an Euler circuit exists — but a Hamilton circuit would have to pass through **v** twice

**Exam trap** "All degrees even" is **necessary and sufficient**, not a shortcut. In a **directed** graph the rule changes: an Euler circuit exists iff the graph is connected and **in-degree = out-degree** at every vertex.

## 3 Hamilton Paths & Circuits

the classic distinction ★

**Definition** A **Hamilton path** visits **every vertex exactly once**; a **Hamilton circuit** closes up. Here it is the **vertices** that count and edges may not repeat. There is no cheap test like Euler's — the problem is NP-complete in general. Useful **sufficient** conditions: **Dirac's theorem** — $\deg(v) \geq n/2$ at every vertex ⇒ Hamilton circuit; **Ore's theorem** — $\deg(u) + \deg(v) \geq n$ for every non-adjacent pair ⇒ Hamilton circuit. Every $K_n$ is Hamiltonian.

**Exam trap — the line that decides the MCQ**

**Euler is about EDGES; Hamilton is about VERTICES.** $K_4$: deg = 3, all **odd** ⇒ **no Euler circuit**, yet it is complete ⇒ **Hamilton circuit ✓**. So **$K_n$ has an Euler circuit exactly when n is odd**. In practice an Euler circuit is the postal-carrier round (polynomial) while a Hamilton circuit is the travelling-salesman problem (NP-complete).

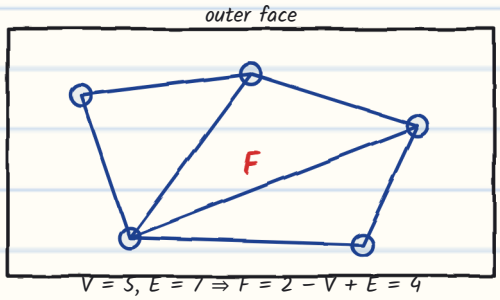
**Worked example ✍ read sheet 2's graph**

$V = \{A, B, C, D, E\}$, edges AB, BC, AD, DE, CE, BD — $\deg(A) = 2$, $\deg(B) = 3$, $\deg(C) = 2$, $\deg(D) = 3$, $\deg(E) = 2$. **(a)** Two odd vertices (B, D) ⇒ an Euler **path**, **no circuit**; it starts at B and ends at D. **(b)** Hamilton path? A → B → C → E → D visits all five once ⇒ **yes**. **(c)** Hamilton circuit? It would need 12 edge-ends = 6 edges, i.e. **every** edge — impossible, since B and D would have to drop from degree 3 to 2 ⇒ **no**.

# 4 Bipartite & Planar Graphs

formula sheet ★

**Definition** A graph is **bipartite** if $V = V_1 \cup V_2$ and every edge joins a vertex of $V_1$ to one of $V_2$ — equivalently, if it can be coloured with **two colours** so that no edge is monochromatic. It is **planar** if it can be drawn in the plane with **no edge crossings**. A bipartite graph contains **no odd cycle**; $K_n$ is bipartite **only for $n \le 2$**, and $K_{3,3}$ is bipartite but not planar.




five vertices, seven edges — the plane is cut into **four** faces, **including the outer one**

**Euler's formula for planar graphs** Connected: $V - E + F = 2$ · with $C$ components: $V - E + F = 1 + C$ · simple planar, $V \ge 3$: $E \le 3V - 6$ · simple planar **and bipartite**, $V \ge 3$: $E \le 2V - 4$.

**Worked example ✍** $V = 5$, $E = 7 \Rightarrow F = 2 - 5 + 7 = 4$. If that graph were **bipartite** and planar, $E \le 2(5) - 4 = 6$, so 7 edges would already be impossible — it has an odd cycle, hence is not bipartite. For $V = 6$ the two ceilings are $3 \cdot 6 - 6 = 12$ and $2 \cdot 6 - 4 = 8$.

**Graph colouring** A **proper colouring** gives adjacent vertices different colours; the fewest colours is the **chromatic number $\chi(G)$**. $\chi(\text{bipartite}) \le 2$; $\chi(K_n) = n$; $\chi(\text{tree}) = 2$. **The four-colour theorem** (Appel & Haken, 1976, computer-assisted): every planar graph is colourable with at most **four** colours, and $K_4$ shows four are really needed. $K_5$ is not planar and needs five.

**Exam trap** (i) "Every planar graph can be coloured with **three** colours" is **false** — that was the disproved claim. (ii) Planarity is a property of **the graph**, not of one bad drawing: $K_5$ cannot be drawn planarly **at all**. (iii) $E \le 3V - 6$ needs $V \ge 3$. (iv) "Connected and $E \ge V$" is exactly the threshold for **containing a cycle**.

**IT link** A **dependency graph** (module A needs module B) is a **digraph**; if it has a cycle the build breaks, so build tools topologically sort it — the DFS of sheet 5. A **routing table** is a weighted graph, and a cheapest route with non-negative costs is exactly Dijkstra.

| Graph class | Edges it can carry | Colours it needs |
|---|---|---|
| Simple planar, $V \ge 3$ | $E \le 3V - 6$; equality when every face is a triangle | $\le 4$ (four-colour theorem) |
| Simple planar **bipartite** | $E \le 2V - 4$; every face has $\ge 4$ sides | $\le 2$ |
| Complete $K_n$ | $n(n-1)/2$ — planar only for $n \le 4$ | exactly $n$ |
| Tree on $n$ vertices | exactly $n - 1$ — always planar, always bipartite | exactly 2 ($n > 1$) |

## 5 Trees

every property asked ★

**Definition** A tree is a **connected acyclic** graph. These are all **equivalent**, and exam questions use them interchangeably: **n vertices ⇒ exactly n − 1 edges** · **exactly one** path between any two vertices · **no cycle** · adding any edge creates a cycle · deleting any edge disconnects it · every degree-1 vertex is a **leaf**, and a tree always has **at least two** leaves.

**Memory trick** "**Tree = n − 1.**" Read it as a **budget**: start from n isolated vertices, i.e. n components; each edge merges exactly two components, so it costs one merge. Going from n components to one therefore takes exactly n − 1 merges — which is also why a connected graph **cannot** have fewer than n − 1 edges.

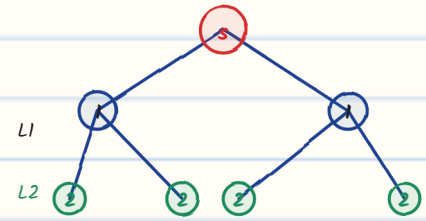
**Exam trap** (i) A tree on n vertices has **n − 1 edges, not n**; a **forest** on n vertices with c components has n − c. (ii) **Acyclic ≠ complete** — a tree has no cycle but also no triangle. (iii) A **rooted** tree adds a chosen root with a **depth**; an **isomorphism** needs a degree sequence **and** the structure to match, so two trees with the same number of edges need not be isomorphic.

## 6 Traversal: BFS & DFS

most tested ★

**Definition** Both **BFS** (breadth-first, a queue) and **DFS** (depth-first, a stack or recursion) visit every reachable vertex once, each returns a **spanning tree** of the component it reached, and **a spanning tree on n vertices always has exactly n − 1 edges**. Both run in O(V + E); only the **order** differs, and that order is the whole answer to "which one do I use".

| | BFS | DFS |
|---|---|---|
| **Structure** | queue — expand level by level | stack / recursion — go as deep as possible |
| **Gives** | **shortest path by edge count** | a DFS forest plus finishing times |
| **Shortest path?** | yes, on an **unweighted** graph | **no**, weighted or not |
| **Used for** | level order, hop counts, nearest node | cycle detection, topological sort |




BFS from **s**: dist(v) is the fewest edges from s to v

**Cycle test** In an undirected graph a **DFS** finds a back edge to a vertex still on the recursion stack ⇒ a cycle exists. In a digraph **BFS** detects a cycle when an already-discovered vertex is met again. Neither test needs weights, so neither gives a **shortest** path on a weighted graph.

**Worked example ✍ V = 6, E = 7** **(a)** Connected? A graph on n vertices with E ≥ n − 1 **may** be connected; here 7 ≥ 5, so connectivity is possible but not guaranteed. **(b)** Cycle? E ≥ V (7 ≥ 6) ⇒ if it is connected it **must** contain a cycle. **(c)** A spanning tree on 6 vertices keeps exactly **5** of those 7 edges.

# 7 Minimum Spanning Tree & Shortest Path

algorithms ★

**Minimum spanning tree** A **spanning tree** connects every vertex with no cycle; an **MST** does it with the smallest total edge weight. Always **n − 1** edges, and **unique only when all weights differ**.

| | Kruskal | Prim |
|---|---|---|
| **Strategy** | sort edges ↑, take each joining two **different** components | grow **one** tree, add the cheapest edge leaving it |
| **Data** | union–find / disjoint sets | a min priority queue |
| **Feeds on** | **sparse** graphs | **dense** graphs |
| **Cost** | O(E log E) | O(E log V) |

**Dijkstra — single-source shortest path** Greedy and correct ~~only when every edge weight is ≥ 0~~. Keep dist[v] for each vertex; repeatedly finalise the unvisited vertex with the smallest dist and **relax** its edges: if dist[u] + w(u,v) < dist[v], set dist[v] = dist[u] + w(u,v) and parent[v] = u. Cost $O(V^2)$, or $O((V+E) \log V)$ with a heap. **Bellman–Ford** relaxes every edge |V| − 1 times, handles **negative weights**, and detects a negative-weight cycle.

**Worked example ✍ Dijkstra on V = {A, B, C, D}** Edges A–B = 4, A–C = 1, C–B = 2, B–D = 3, C–D = 5. **(1)** dist = (A 0, B 4, C 1, D ∞). Smallest unsettled is **C** at 1; settle it. Relax C–B: 1 + 2 = 3 < 4 ⇒ dist[B] = **3**. Relax C–D: 1 + 5 = 6 ⇒ dist[D] = **6**. **(2)** Smallest unsettled is **B** at 3; settle it. Relax B–D: 3 + 3 = 6, not < 6 ⇒ dist[D] stays **6** (a tie — the route A–C–D also costs 6). **(3)** Settle **D** at 6. Final: ~~d(A) = 0, d(C) = 1, d(B) = 3, d(D) = 6~~, with parents C ← A, B ← C. **Kruskal on the same graph:** sorted AC(1), BC(2), BD(3), AB(4), CD(5) — take **AC**, then **BC**, then **BD**. Three edges = n − 1, no cycle ⇒ an MST of weight 1 + 2 + 3 = **6**.

**Exam trap** Dijkstra finds a **shortest path**; Kruskal and Prim find a **minimum spanning tree**. They are different questions and on a general graph they choose different edges. An MST exists only for a **connected** graph — on a disconnected one both algorithms return a **minimum spanning forest**.

**Memory trick** "**K**ruskal goes **K**ayaking over **K**rossed-out edges" — take the cheapest, throw away anything that would close a loop. "**P**rim stays **P**ut" — it never leaves the tree it has already grown. And "**negative** breaks **D**ijkstra" — the moment a cheaper path lies behind you, a settled vertex is never revisited.

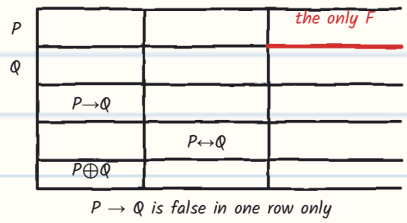
**Why Dijkstra breaks on a negative edge — the one-line proof** Dijkstra declares a vertex **final** when it is cheapest among the unsettled ones. If a later edge could still improve that vertex, it must run through some unsettled vertex whose distance is **not smaller** — impossible when every weight added is ≥ 0. A single negative edge destroys exactly that inequality, so the "final" verdict is wrong.

# 8 Propositional Calculus: Connectives

foundation ★

**Definition** A **proposition** is a declarative sentence that is **either true or false, and nothing else** — no "maybe", no dependence on an unknown. "2 + 2 = 4" is a proposition; "x + 2 = 5" is an **open sentence**, not a proposition until x is given a value. A **compound** proposition is built from simple ones with **logical connectives**.

| Connective | Symbol | Reads | True exactly when |
|---|---|---|---|
| **Negation** | $\neg P$ | not P | P is **false** ($T \to F$, $F \to T$) |
| **Conjunction** | $P \wedge Q$ | P and Q | **both** are true |
| **Disjunction** | $P \vee Q$ | P or Q | **at least one** is true (inclusive) |
| **Exclusive or** | $P \oplus Q$ | P or Q, not both | **exactly one** is true |
| **Implication** | $P \to Q$ | if P then Q | P is false, **or** Q is true |
| **Biconditional** | $P \leftrightarrow Q$ | P if and only if Q | **both have the same truth value** |



$P \to Q$ is false in one row only

| P | Q | P∧Q | P∨Q | P⊕Q | P→Q | P↔Q |
|---|---|---|---|---|---|---|
| T | T | T | T | **F** | T | T |
| T | F | **F** | T | T | **F** | **F** |
| F | T | **F** | T | T | T | **F** |
| F | F | **F** | **F** | **F** | T | T |

the implication column is F **exactly once** — that single row is what every MCQ about $p \to q$ turns on

**Three classifications** **Tautology** — always T ($P \vee \neg P$, $P \to P$) · **Contradiction** — always F ($P \wedge \neg P$, $P \oplus P$) · **Contingency** — T in some rows and F in others ($P \vee Q$). **Test: build the column; if every entry matches, it is one of the first two.**

**Exam trap** (i) **$p \to q$ is false only at $p$ = T, $q$ = F**; the other three rows are true, so a false hypothesis still yields a true implication. (ii) $P \oplus Q$ is true when **exactly one** is true — with both true it is **false**. (iii) **$P \leftrightarrow Q$ is an operator; $P \equiv Q$ states equivalence** — the first is built from P and Q, the second compares two whole formulas. (iv) **Precedence** $\neg > \wedge > \vee > \to > \leftrightarrow$, so $\neg P \wedge Q$ means $(\neg P) \wedge Q$.

**Memory trick** "**M**inus **A**gain **M**akes **N**othing" — the one connective that turns a false hypothesis into a true conclusion. "**xor is nothing-or-something**": $P \oplus Q \equiv \neg(P \leftrightarrow Q)$. Implication is the only connective that is not symmetric, while $\leftrightarrow$ and $\equiv$ are the only genuine equivalences.

**Building a truth table — the routine that earns the marks** Write the formula with parentheses already inserted by precedence. One row per combination: n variables give **$2^n$ rows** and the column order must be **fixed** across all of them. Fill each column top to bottom. Then read the last column: all T = tautology, all F = contradiction, mixed = contingency. For $2^n > 8$ rows, first push everything to **CNF** or use the laws instead of the table.

# 9 Equivalence, Laws & Canonical Forms

written favourite ★

**Definition** Two formulas **P** and **Q** are **logically equivalent**, written **P ≡ Q**, when they have the **same truth value in every row** — equivalently when **P ↔ Q is a tautology**. A truth table proves it; the laws below prove it in one line.

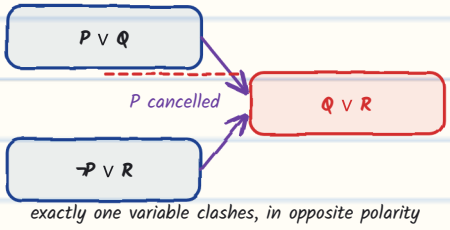
| Law | Statement |
|---|---|
| **Identity** | $P \wedge T \equiv P$, $P \vee F \equiv P$ |
| **Domination** | $P \wedge F \equiv F$, $P \vee T \equiv T$ |
| **Idempotent** | $P \wedge P \equiv P$, $P \vee P \equiv P$ |
| **Complement** | $P \wedge \neg P \equiv F$, $P \vee \neg P \equiv T$, $\neg\neg P \equiv P$ |
| **Commutative** | $P \wedge Q \equiv Q \wedge P$, $P \vee Q \equiv Q \vee P$ |
| **Associative** | $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$ — and for $\vee$ |
| **Distributive** | $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$; $P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ |
| **De Morgan** | $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$, $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ |
| **Absorption** | $P \vee (P \wedge Q) \equiv P$, $P \wedge (P \vee Q) \equiv P$ |

**The five forms to know by heart** $P \to Q \equiv \neg P \vee Q$ · $\neg Q \to \neg P \equiv P \to Q$ (the **contrapositive**) · $Q \to P$ is the **converse** and $\neg P \to \neg Q$ the **inverse**, neither equivalent, though **converse ≡ inverse** · $P \leftrightarrow Q \equiv (P \wedge Q) \vee (\neg P \wedge \neg Q) \equiv (P \to Q) \wedge (Q \to P)$ · $P \oplus Q \equiv \neg(P \leftrightarrow Q)$.

**Worked example ✍ exportation** $P \to (Q \to R) \equiv \neg P \vee (\neg Q \vee R) \equiv (\neg P \vee \neg Q) \vee R \equiv \neg(P \wedge Q) \vee R \equiv (P \wedge Q) \to R$. With $P = F$, $Q = F$, $R = F$ the chained form $(P \to Q) \to R$ is **false** while the original is **true**, so **implication is the one connective that is not associative**.

**Canonical (normal) forms** A formula in **DNF** (sum of products) is a **disjunction of conjunctions**; in **CNF** (product of sums) it is a **conjunction of disjunctions**. Every formula has **both**, read off its truth table by taking exactly **one minterm per row**: DNF takes the rows where it is true, CNF the rows where it is false.

~~**DNF of $P \to Q$ is $(\neg P) \vee (P \wedge Q)$; CNF is $(\neg P \vee Q)$.**~~




resolution: cancel the shared variable, keep the rest

**Resolution — the one automated rule** It applies to **CNF only**. From $(P \vee Q)$ and $(\neg P \vee R)$ cancel $P$ and take the **resolvent** $(Q \vee R)$. Resolution is **sound** — it never draws a false conclusion — but **not complete** on non-CNF input, which is why automated theorem provers convert everything to CNF first.

1. Write every premise as a clause, i.e. in CNF.
2. Find two clauses sharing one variable in opposite polarity.
3. Drop that variable and join the rest — the resolvent; the empty clause □ means the premises contradict.

## 10 Predicate Calculus: Quantifiers

most tested ★

**Definition** Predicate logic adds **variables**, **predicates** and **quantifiers** to propositional logic, so it can speak about a statement's **internal structure**. A predicate $P(x)$ is a property of $x$; the set of values $x$ may take is its **domain**. A variable is **bound** if a quantifier covers it and free otherwise: only a bound formula is a proposition.

| Words | Formula | Read |
|---|---|---|
| For every / there exists | $\forall x\ P(x)$ · $\exists x\ P(x)$ | $P(x)$ holds for **every** $x$, or for **at least one** $x$ |
| Every … some, vs some … every | $\forall x\ \exists y$ vs $\exists x\ \forall y\ P(x, y)$ | the $y$ **may depend** on $x$, or one $x$ works for all $y$ |

**The four quantifier laws** $\neg\forall x\ P(x) \equiv \exists x\ \neg P(x)$ · $\neg\exists x\ P(x) \equiv \forall x\ \neg P(x)$ · $\forall x\ (P \wedge Q) \equiv (\forall x\ P) \wedge (\forall x\ Q)$ · $\exists x\ (P \vee Q) \equiv (\exists x\ P) \vee (\exists x\ Q)$ — while $\forall x\ (P \vee Q) \not\equiv (\forall x\ P) \vee (\forall x\ Q)$ and its dual.

**Exam trap** (i) "All humans are mortal" is $\forall x\ (\text{Human}(x) \rightarrow \text{Mortal}(x))$ — with an **implication**, never $\forall x(\text{Human} \wedge \text{Mortal})$, which would absurdly claim every object is human. (ii) "There is a prime greater than 100" is $\exists x\ (\text{Prime}(x) \wedge x > 100)$. (iii) $\forall x\ \exists y \neq \exists x\ \forall y$ — "everyone has a friend" is the first. In SQL, $\forall x\ P(x)$ is SELECT … WHERE with no GROUP BY, $\exists x\ P(x)$ is EXISTS, and $\forall x\ \exists y$ is a correlated subquery returning a different $y$ per row while $\exists x\ \forall y$ forces one row that works for all.

## 11 Rules of Inference

written favourite ★

**Definition** An argument is a set of **premises** followed by a **conclusion**; it is **valid** when no truth-table row has every premise T and the conclusion F.

~~Valid ⇒ the conclusion is true; a true conclusion ⇏ a valid argument.~~

| Rule | Form | Rule | Form |
|---|---|---|---|
| Modus ponens | $P \rightarrow Q,\ P\ \therefore\ Q$ | Addition | $P\ \therefore\ P \vee Q$ |
| Modus tollens | $P \rightarrow Q,\ \neg Q\ \therefore\ \neg P$ | Simplification | $P \wedge Q\ \therefore\ P$ |
| Hyp. syllogism | $P \rightarrow Q,\ Q \rightarrow R\ \therefore\ P \rightarrow R$ | Conjunction | $P, Q\ \therefore\ P \wedge Q$ |
| Disj. syllogism | $P \vee Q,\ \neg P\ \therefore\ Q$ | Resolution | $P \vee Q,\ \neg P \vee R\ \therefore\ Q \vee R$ |
| UI · UG · EI · EG | UI: $\forall x\ P(x) \therefore P(a)$ · UG: $P(a) \therefore \forall x\ P(x)$ · EI: $\exists x\ P(x) \therefore P(c)$ · EG: $P(c) \therefore \exists x\ P(x)$ | | |

**Exam trap — the classic fallacies**

**Affirming the consequent:** $P \rightarrow Q,\ P \therefore Q$ — **invalid** ("if it rains the road is wet" + "the road is wet" does not prove rain). **Denying the antecedent:** $P \rightarrow Q,\ \neg P \therefore \neg Q$ — **invalid**. **Undistributed middle:** all $A$ are $B$, all $C$ are $B$ $\therefore$ all $A$ are $C$ — invalid.

**Worked examples** ✍ **(a)** $\forall x(H(x) \rightarrow M(x))$ with $H(s)$: **UI** then **modus ponens** ⇒ $M(s)$ **(b)** Premises $(P \vee Q)$, $(\neg P \vee R)$, $(\neg R)$: resolve the last two ⇒ $\neg P$, with the first ⇒ $Q$.

# Night-Before-the-Exam Sheet

## Fact sheet

**Σdeg = 2|E|** · odd vertices in pairs
$K_n$: deg n−1, edges n(n−1)/2
Euler circuit ⇔ all deg **even**
Euler path ⇔ exactly **2** odd
Euler = **edges**, Hamilton = **vertices**
$K_n$ Euler circuit ⇔ n **odd**
Dirac: deg ≥ n/2 ⇒ Hamilton
Cycle ⇒ E ≥ V; connected ⇒ E ≥ V−1
Planar: **V − E + F = 2**
Simple planar: E ≤ **3V − 6**
Bipartite planar: E ≤ **2V − 4**
Four-colour theorem; $\chi(K_n) = n$
Tree: n vertices ⇒ **n − 1** edges
Tree ⇒ unique path, no cycle
BFS = queue = **shortest** hops
DFS = stack = **topological sort**
BFS/DFS cost O(V + E)
Spanning tree always n − 1 edges
Dijkstra needs w ≥ 0
Bellman–Ford: negatives + cycle detect
Kruskal: sort, union–find, O(E log E)
Prim: one tree, heap, O(E log V)
p → q false only at T, F
$P \oplus Q \equiv \neg(P \leftrightarrow Q)$
p → q ≡ ¬p ∨ q
Contrapositive ¬q → ¬p
¬∀x ≡ ∃x¬ · ¬∃x ≡ ∀x¬
All humans mortal = ∀x(H → M)
UI, UG, EI, EG
Resolution: cancel one variable

## → Top 12 one-liners for MCQs

1. Sum of degrees = **2 × edges**
2. Odd-degree vertices are **even in number**
3. Euler circuit ⇔ every degree **even**
4. Euler counts **edges**, Hamilton counts **vertices**
5. Planar: **V − E + F = 2**
6. A tree on n vertices has **n − 1** edges
7. BFS finds shortest **unweighted** paths
8. DFS gives topological sorts
9. Dijkstra breaks on **negative weights**
10. **p → q** is false only at T, F
11. De Morgan **flips** ∧ into ∨
12. **¬∀x P ≡ ∃x ¬P**

**Top traps** "cycle = starts and ends at the same vertex" without saying **no other vertex repeats** · a tree on **n** vertices has **n** edges, not **n−1** · "every planar graph is 3-colourable" · Euler's formula with the outer face **left out** · claiming a graph is bipartite from one bad drawing · Dijkstra on negative weights · **p ⊕ q** true when both are true · treating ↔ and ≡ as the same symbol · ∀x(P∨Q) ≡ (∀xP)∨(∀xQ) · affirming the consequent.

**All tricks together** "Walks are wild, trails have tight edges, paths are pure" · "Euler walks every street, Hamilton visits every city" · "cycle needs length ≥ 3" · "two ends per edge — count the ends" · "tree = n − 1" · "Kruskal kayaking, cross out the loops; Prim stays put" · "minuses again makes nothing" · "xor is nothing-or-something" · "minus again, or exists" · "modus ponens pokes, modus tollens takes away".

**VALID**
no row: all premises T, conclusion F
modus ponens, resolution, UI

**INVALID**
affirming the consequent, denying the antecedent, undistributed middle

one truth-table row separates them

the whole inference idea in two boxes

## ✓ Practice MCQs (exam pattern, 10 marks)

try first, then check

**1.** An undirected graph has 7 edges. The sum of all vertex degrees is:
(a) 14 (b) 7
(c) 21 (d) 49

**2.** The number of edges in $K_6$ is:
(a) 30 (b) 12
(c) 15 (d) 36

**3.** If every vertex of a graph has degree 3, the number of vertices is:
(a) odd (b) even
(c) prime (d) at least 5

**4.** A connected planar graph with $V = 5$ and $E = 8$ has how many faces?
(a) 4 (b) 6
(c) 8 (d) 5

**5.** $K_n$ has an Euler circuit exactly when $n$ is:
(a) odd (b) even
(c) prime (d) at least 3

**6.** Dijkstra's algorithm fails to give correct shortest paths when the graph has:
(a) unit weights (b) negative edge weights
(c) undirected edges (d) two sources

**7.** The negation of $\forall x\, P(x)$ is:
(a) $\forall x\, \neg P(x)$ (b) $\exists x\, P(x)$
(c) $\neg\forall x\, \neg P(x)$ (d) $\exists x\, \neg P(x)$

**8.** The contrapositive of $p \to q$ is:
(a) $q \to p$ (b) $\neg p \to \neg q$
(c) $\neg q \to \neg p$ (d) $p \to \neg q$

**9.** A simple bipartite planar graph on $V$ vertices has at most:
(a) $3V - 6$ edges (b) $V^2$ edges
(c) $2V - 4$ edges (d) $2V - 2$ edges

**10.** The minimum edges a connected graph with $n$ vertices can have:
(a) $n - 1$ (b) $n$
(c) $n + 1$ (d) $\lceil n/2 \rceil$

Answer key → 1-a · 2-c · 3-b · 4-d · 5-a · 6-b · 7-d · 8-c · 9-c · 10-a

**Written question** A simple undirected graph $G$ has 6 vertices, 9 edges and every vertex of degree 3. (i) Verify the handshaking lemma. (ii) Can you conclude that $G$ is bipartite? Justify. (iii) A connected planar graph has $V = 5$ and $E = 8$; find $F$, and the largest $E$ for a bipartite planar graph on 5 vertices. (iv) Distinguish a proposition from a predicate and negate $\forall x\, P(x) \vee \exists x\, Q(x)$. 10

1. **Handshaking** — $\Sigma \deg(v) = 2|E| = 2 \times 9 = \mathbf{18}$, and 6 vertices of degree 3 give $6 \times 3 = 18$ ✓. 2
2. **Bipartite?** Cannot be determined. $K_{3,3}$ has the same data and is bipartite; the triangular prism has the same data but a triangle, so it is not. 3
3. **Planar counts** — $F = 2 - V + E = 2 - 5 + 8 = \mathbf{5}$, the outer face included. Bipartite planar on $V = 5$: $E \le 2V - 4 = \mathbf{6}$. 3
4. **Logic** — a **proposition** has one definite truth value; a **predicate** $P(x)$ becomes one only once $x$ is bound. $\neg(\forall x\, P(x) \vee \exists x\, Q(x)) \equiv \exists x\, \neg P(x) \wedge \forall x\, \neg Q(x)$. 2

### → Before the exam ✓

- ✓ Apply $\Sigma\deg = 2|E|$ without hesitating
- ✓ Decide Euler vs Hamilton from the object counted
- ✓ Fill a truth table for any 2- or 3-variable formula
- ✓ Apply $\neg\forall \equiv \exists\neg$ and the valid inference rules